EX No.: 5

Date:

Study of Network simulator

Aim:

To study Network Simulator in detail.

Introduction:

Network Simulator (Version 2), widely known as NS2, is simply an event driven simulation tool that has proved useful in studying the dynamic nature of communication networks. Simulation of wired as well as wireless network functions and protocols (e.g., routing algorithms, TCP, UDP) can be done using NS2. In general, NS2 provides users with a way of specifying such network protocols and simulating their corresponding behaviours. Due to its flexibility and modular nature, NS2 has gained constant popularity in the networking research community since its birth in 1989. Ever since, several revolutions and revisions have marked the growing maturity of the tool, thanks to substantial contributions from the players in the field. Among these are the University of California and Cornell University who developed the REAL network simulator, 1 the foundation which NS is based on.

Since 1995 the Defence Advanced Research Projects Agency (DARPA) supported development of NS through the Virtual Inter Network Testbed (VINT) project currently the National Science Foundation (NSF) has joined the ride in development. Last but not the least, the group of Researchers and developers in the community are constantly working to keep NS2 strong and versatile.

Basic Architecture:

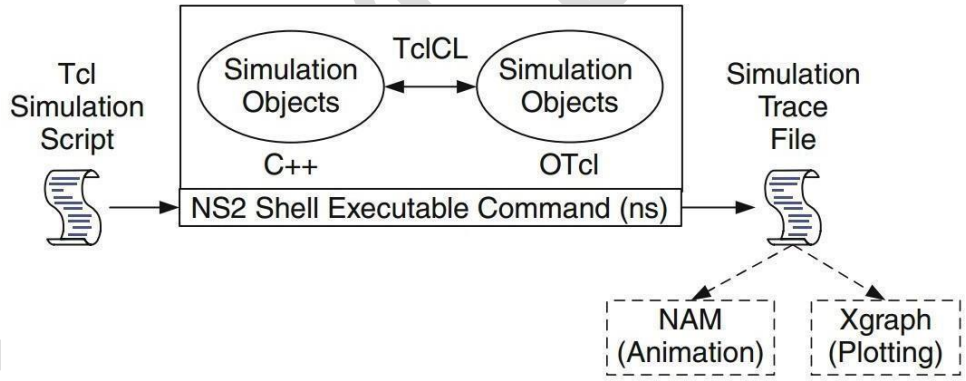


Fig. 2.1. Basic architecture of NS.

Figure shows the basic architecture of NS2. NS2 provides users with executable command ns which take on input argument, the name of a Tcl simulation scripting file. Users are feeding the name of a Tcl simulation script (which sets up a simulation) as an input argument of an NS2 executable command ns.

In most cases, a simulation trace file is created, and is used to plot graph and/or to create animation. NS2 consists of two key languages: C++ and Object-oriented Tool Command Language (OTcl). While the C++ defines the internal mechanism (i.e., a backend) of the simulation objects, the OTcl sets up simulation by assembling and configuring the objects as well as scheduling discrete events (i.e., a frontend).

The C++ and the OTcl are linked together using TclCL. Mapped to a C++ object, variables in the OTcl domains are sometimes referred to as handles. Conceptually, a handle (e.g., n as a Node handle) is just a string (e.g., _o10) in the OTcl domain, and does not contain any functionality. Instead, the functionality (e.g., receiving a packet) is defined in the mapped C++ object (e.g., of class Connector). In the OTcl domain, a handle

acts as a frontend which interacts with users and other OTcl objects. It may define its own procedures and variables to facilitate the interaction. Note that the member procedures and variables in the OTcl domain are called instance procedures (instprocs) and instance variables (instvars), respectively. Before proceeding further, the readers are encouraged to learn C++ and OTcl languages.

NS2 provides a large number of built-in C++ objects. It is advisable to use these C++ objects to set up a simulation using a Tcl simulation script. However, advance users may find these objects insufficient. They need to develop their own C++ objects, and use a OTcl configuration interface to put together these objects. After simulation, NS2 outputs either text-based or animation-based simulation results. To interpret these results graphically and interactively, tools such as NAM (Network AniMator) and XGraph are used. To analyze a particular behavior of the network, users can extract a relevant subset of text-based data and transform it to a more conceivable presentation.

Concept Overview:

ns uses two languages because simulator has two different kinds of things it needs to do. On one hand, detailed simulations of protocols require a systems programming language which can efficiently manipulate bytes, packet headers and implement algorithms that run over large data sets. For these tasks run-time speed is important and turn-around time (run simulation, find bug, fix bug, recompile, re-run) is less important. On the other hand, a large part of network research involves slightly varying parameters or configurations, or quickly exploring number of scenarios.

In these cases, iteration time (change the model and re-run) is more important. Since configuration runs once (at the beginning of the simulation), run-time of this part of the task is less important. ns meets both of these needs with two languages, C++ and OTcl.

Basic Commands in Ns2:

- Create event scheduler
  - **set ns [new Simulator]**
- Trace packets on all links
  - **set nf [open out.nam w]**
  - **$ns trace-all $nf**
  - **$ns namtrace-all $nf**
- Nodes
  - **set n0 [$ns node]**
  - **set n1 [$ns node]**
- Links and queuing
  - $ns duplex-link $n0 $n1 <bandwidth> <delay> <queue_type>
  - <queue_type>: DropTail, RED, etc.
  - **$ns duplex-link $n0 $n1 1Mb 10ms RED**
- Creating a larger topology

```
for {set i 0} {$i < 7} {incr i} {
set n($i) [$ns node]
}
for {set i 0} {$i < 7} {incr i} {
$ns duplex-link $n($i) $n([expr ($i+1)%7]) 1Mb 10ms RED
}
```

3112410015

- Link failures
  - **$ns rtmodel-at <time> up|down $n0 $n1**
- Creating UDP connection
  - **set udp [new Agent/UDP]**
  - **set null [new Agent/Null]**
  - **$ns attach-agent $n0 $udp**
  - **$ns attach-agent $n1 $null**
  - **Sns connect $udp $null**
- Creating Traffic (On Top of UDP)
  - set cbr [new Application/Traffic/CBR]
  - $cbr set packetSize 500
  - $cbr set interval 0.005
  - $cbr attach-agent $udp
- Post-Processing Procedures

```
proc finish {}
{
global ns nf
$ns flush-trace
close $nf
exec nam out.nam &
exit 0
}
```

- Schedule Events
  - **$ns at <time> <event>**
- Call 'finish'
  - **$ns at 5.0 "finish"**
- Run the simulation
  - **$ns run**

311241040 15

Result:

Thus the study of Network Simulator was done successfully.

EX No.: 6

Date: **Simulation of Congestion Control Algorithms using NS**

Aim:

To perform simulation of congestion control algorithms (sliding window) using NS2, entirely from the Ubuntu terminal.

Setup:

sudo apt update
sudo apt install ns2
optional, for plotting the window size
sudo apt install gnuplot

If apt cannot find ns2, enable the universe repository first: sudo add-apt-repository universe, then run sudo apt update again. Check the install with which ns.

Algorithm:

1. Create a simulator object and open the trace file sliding. tr and the window log file cwnd.tr.
2. Create two nodes n0 (sender) and n1 (receiver).
3. Create a duplex link between them: DropTail queue, 0.2 Mbps capacity, 200ms propagation delay, queue size 10.
4. Create a TCP agent on n0 and a TCP sink on n1, and connect them. Set packet size 500 bytes, initial window 4 and maximum window 4.
5. Attach an FTP application to the TCP agent. Start at 0.1 s and stop at 3.5 s.
6. Record cwnd_ every 0.1s.
7. End the simulation at 5.0 s, flush and close the files, and run.

Program:

```
set val(stop) 5.0
set ns [new Simulator]
set tracefile [open sliding.tr w]
$ns trace-all $tracefile
set cwndfile [open cwnd.tr w]
set n0 [$ns node]
set n1 [$ns node]
$ns duplex-link $n0 $n1 0.2Mb 200ms DropTail
$ns queue-limit $n0 $n1 10
set tcp1 [new Agent/TCP]
$tcp1 set windowInit_ 4
$tcp1 set maxcwnd_ 4
$tcp1 set packetSize_ 500
$ns attach-agent $n0 $tcp1
set sink2 [new Agent/TCPSink]
$ns attach-agent $n1 $sink2
```

2112410401

```
$ns connect $tcp1 $sink2
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp1
$ns at 0.1 "$ftp0 start"
$ns at 3.5 "$ftp0 stop"
proc record {} {
global ns tcp1 cwndfile
puts $cwndfile "[$ns now] [$tcp1 set cwnd_]"
$ns at [expr [$ns now] + 0.1] "record" }
$ns at 0.0 "record"
proc finish {} {
global ns tracefile cwndfile
$ns flush-trace
close $tracefile
close $cwndfile
puts "Simulation finished."
puts "Packet trace : sliding.tr"
puts "Window log : cwnd.tr"
exit 0
}
$ns at $val(stop) "finish"
$ns run
```

Steps to Execute:

1. Open a terminal and go to the folder where the file is saved.
   mkdir -p ~/cn_lab && cd ~/cn_lab
   nano sliding.tcl #write the program, save with Ctrl+O, exit with Ctrl+X
2. Run the simulation.
   ns sliding.tcl
3. Check that sliding.tr and cwnd.tr were created.
   ls -l sliding.tr cwnd.tr

Examine the Output:

Each line of sliding.tr has the format: event (+ enqueue, - dequeue, r receive, d drop), time, from node, to node, packet type, size, flags, flow id, source, destination, sequence number, packet id.

Packets sent by the sender (node 0):

```
awk '$1=="+" && $3==0 && $5=="tcp" {print "Send Packet", $11, "at", $2}' sliding.tr |
head -20
```

Acknowledgements received by the sender:

```
awk '$1=="r" && $4==0 && $5=="ack" {print "Receive Ack", $11, "at", $2}' sliding.tr |
head -20
```

Packets dropped:

```
awk '$1=="d"' sliding.tr | wc -l
```

3112410401 5

Data received at the receiver and approximate throughput:

awk '$1=="r" && $4==1 && $5=="tcp" {n++; bytes+=$6} END {print n, "packets,", bytes, "bytes,", bytes*8/3.4/1000, "kbps"}' sliding.tr

Window size over time:

cat cwnd.tr

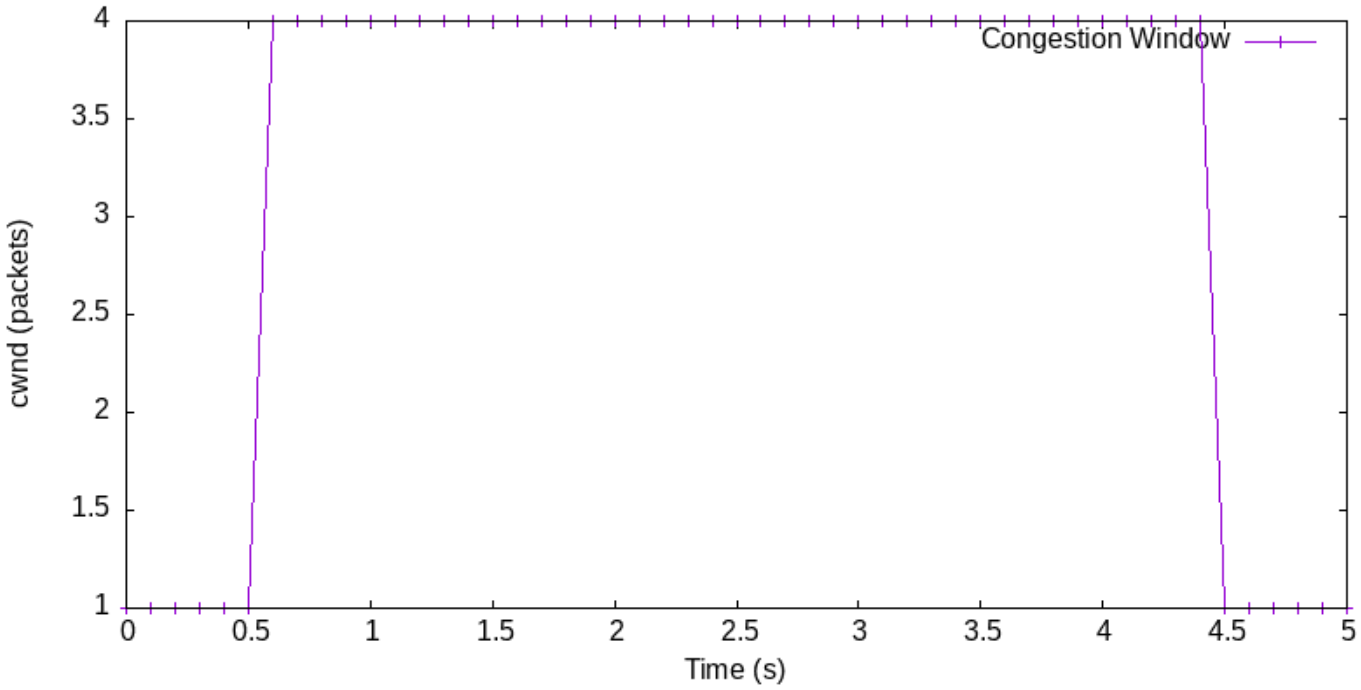
Optional plot of the window size (saves a PNG):

gnuplot -e "set terminal png size 800,400; set output 'cwnd.png'; set xlabel 'Time (s)'; set ylabel 'cwnd (packets)'; plot 'cwnd.tr' with linespoints title 'cwnd'"

Output:

```
asen@asen:~/cn_lab$ ns sliding.tcl
Simulation finished.
Packet trace :sliding.tr
Window log : cwnd.tr
asen@asen:~/cn_lab$ awk '$1=="+" && $3==""0" && $5=="tcp" {print "Send Packet", $11, "at", $2}' sliding.tr | head -20
Send Packet 0 at 0.1
Send Packet 1 at 0.5032
Send Packet 2 at 0.5032
Send Packet 3 at 0.5032
Send Packet 4 at 0.5032
Send Packet 5 at 0.9264
Send Packet 6 at 0.948
Send Packet 7 at 0.9696
Send Packet 8 at 0.9912
Send Packet 9 at 1.3496
Send Packet 10 at 1.3712
Send Packet 11 at 1.3928
Send Packet 12 at 1.4144
Send Packet 13 at 1.7728
Send Packet 14 at 1.7944
Send Packet 15 at 1.816
Send Packet 16 at 1.8376
Send Packet 17 at 2.196
Send Packet 18 at 2.2176
Send Packet 19 at 2.2392
```

```
Receive Ack 0 at 0.5032
Receive Ack 1 at 0.9264
Receive Ack 2 at 0.948
Receive Ack 3 at 0.9696
Receive Ack 4 at 0.9912
Receive Ack 5 at 1.3496
Receive Ack 6 at 1.3712
Receive Ack 7 at 1.3928
Receive Ack 8 at 1.4144
Receive Ack 9 at 1.7728
Receive Ack 10 at 1.7944
Receive Ack 11 at 1.816
Receive Ack 12 at 1.8376
Receive Ack 13 at 2.196
Receive Ack 14 at 2.2176
Receive Ack 15 at 2.2392
Receive Ack 16 at 2.2608
Receive Ack 17 at 2.6192
Receive Ack 18 at 2.6408
Receive Ack 19 at 2.6624
```



3111241042

Result:

Thus, the simulation of congestion control (sliding window) was performed using NS2 from the Ubuntu terminal.

EX No.: 7.

Date: **Wireshark to capture packets**

Aim:

To use a tool like Wireshark to capture packets and examine the packets and capture ping and tracert PDUs and various protocol PDUs.

Setup:

sudo apt update

sudo apt install wireshark

Algorithm:

Step-1: Start.

Step-2: Install and open Wireshark in Ubuntu.

Step-3: Select the active network interface and start capturing packets.

Step-4: Open the Terminal and execute the ping command to generate ICMP packets.

Step-5: Apply the ICMP filter in Wireshark and examine the captured packets.

Step-6: Execute the traceroute command to capture network hop packets.

Step-7: Apply the ICMP and UDP filter and examine the captured packets.

Step-8: Generate DNS, TCP, UDP and ARP traffic using suitable commands.

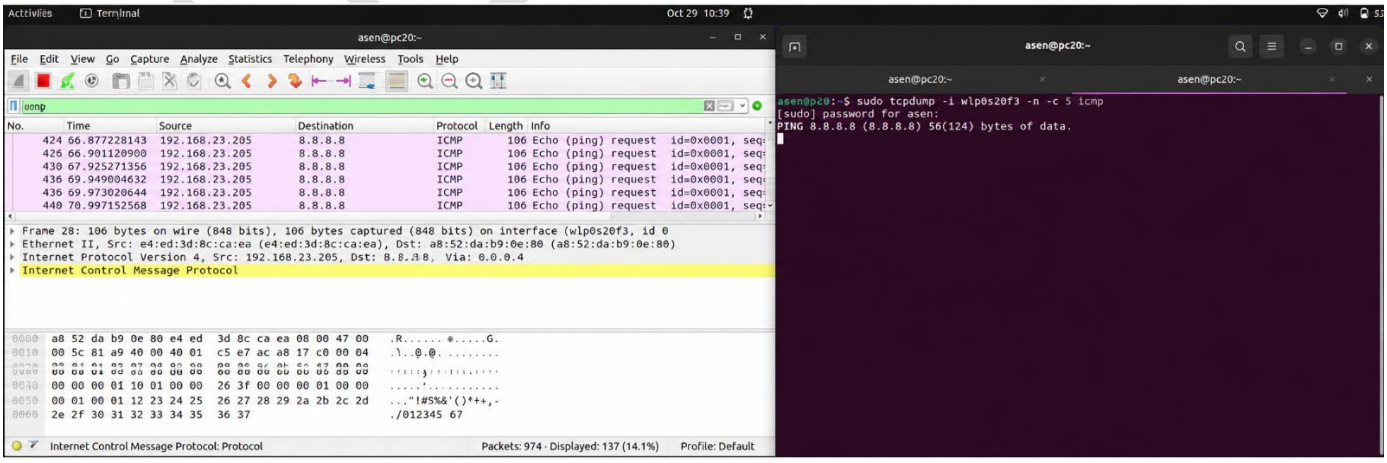
Step-9: Apply the corresponding protocol filters and examine the packet details.

Step-10: Stop the capture and save the captured packets in a file.

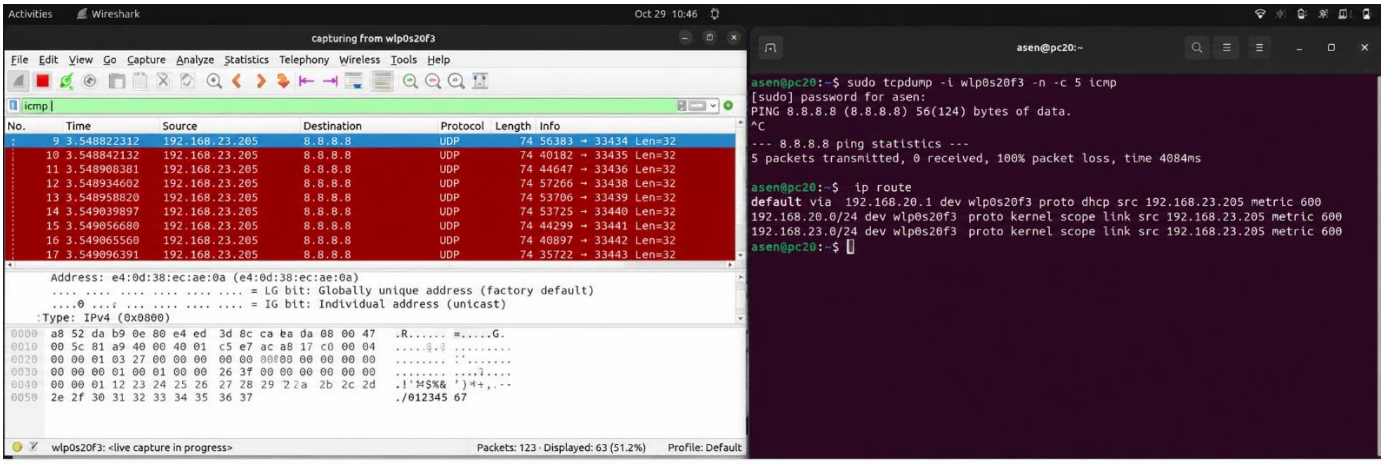
Step-11: Stop.

Output:

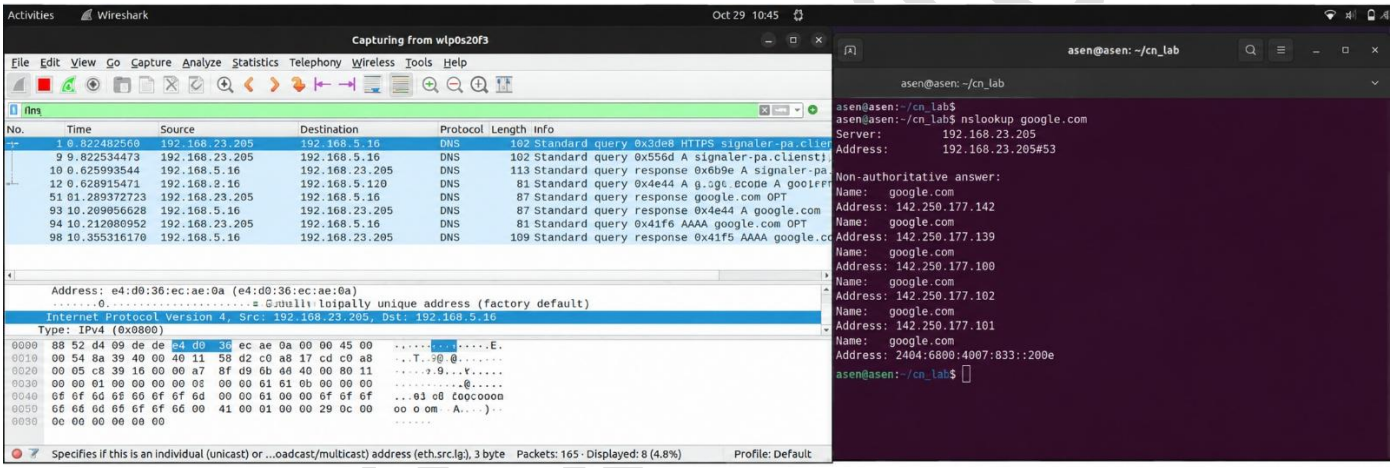
1. Capture Ping Packets
   a. **ping -c 4 8.8.8.8**
   b. **In wireshark, apply filter "icmp"**



2. Capture Traceroute Packets
    a. **sudo traceroute 8.8.8.8**
    b. **In wireshark apply filter “icmp || udp”**



3. Capture DNS Packets
    a. **nslookup google.com**
    b. **In wireshark apply filter “dns”**



4. Caputre TCP Packets
    a. **curl http://google.com**
    b. **In wireshark apply filter “tcp”**

Result:

Thus, used a tool like Wireshark to capture packets and examine the packets like ping, traceroute, dns, tcp.

EX No.: 8. a)

Date: **Applications using TCP sockets like Echo client and echo server**

Aim:

To write a program in Java to implement applications using TCP sockets like echo client and echo server.

Algorithm:

1. Start the program.
2. In Server
    a. Create a server socket and bind it to a port.
    b. Listen for a new connection and, when a connection arrives, accept it.
    c. Read the data from the client.
    d. Echo the data back to the client.
    e. Close all streams.
    f. Close the server socket.
    g. Stop.
3. In Client
    a. Create a client socket and connect it to the server's port number.
    b. Send user data to the server.
    c. Display the data echoed by the server.
    d. Close the input and output streams.
    e. Close the client socket.
    f. Stop.
4. Stop the program.

Program: Server.java

```
import java.net.*;
import java.io.*;

public class Server {
   public static final int PORT = 4000;

   public static void main(String args[]) {
      ServerSocket sersock = null;
      Socket sock = null;
      try {
         sersock = new ServerSocket(PORT);
         System.out.println("Server Started :" + sersock);
         try {
            sock = sersock.accept();
            System.out.println("Client Connected :" + sock);
            System.out.println(" Connection from : " + sock.getInetAddress());
```

3112410401 5

```
            BufferedReader ins = new BufferedReader(new
      InputStreamReader(sock.getInputStream()));
            System.out.println(ins.readLine());
            PrintStream ios = new PrintStream(sock.getOutputStream());
            ios.println("Hello from server");
            ios.close();
            sock.close();
          } catch (SocketException se) {
            System.out.println("Server Socket problem " + se.getMessage());
          }
        } catch (Exception e) {
          System.out.println("Couldn't start " + e.getMessage());
        } finally {
          try {
            if (sersock != null) {
              sersock.close();
            }
          } catch (IOException e) {
            System.out.println("Close Error: " + e.getMessage());
          }
        }
      }
    }
```

Program: Client.java

```
    import java.io.*;
    import java.net.*;
    import java.net.InetAddress;
    class Client {
      public static final int PORT = 4000;
      public static void main(String args[]) {
        Socket sock = null;
        PrintStream ps = null;
        System.out.println(" Trying to connect");
        try {
          sock = new Socket(InetAddress.getLocalHost(), PORT);
          ps = new PrintStream(sock.getOutputStream());
          ps.println(" Hi from client");
          BufferedReader is = new BufferedReader(new InputStreamReader(sock.getInputStream()));
          System.out.println(is.readLine());
        } catch (SocketException e) {
          System.out.println("SocketException " + e);
        } catch (IOException e) {
          System.out.println("IOException " + e);
```

3112410401 5

```
        } finally {
            try {
                if (sock != null) {
                    sock.close();
                }
            } catch (IOException ie) {
                System.out.println(" Close Error :" + ie.getMessage());
            }
        }
    }
}
```

Output:

```
E:\asen_CN>javac Server.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

E:\asen_CN>java Server.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Server Started :ServerSocket[addr=0.0.0.0/0.0.0.0,localport=4000]
Client Connected :Socket[addr=/172.31.99.190,port=56180,localport=4000]
 Connection from : /172.31.99.190
 Hi from client

E:\asen_CN>
```

```
E:\asen_CN>javac Client.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

E:\asen_CN>java Client.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
 Trying to connect
Hello from server

E:\asen_CN>
```

Result:

Thus, a program in Java implemented an application using TCP sockets like echo client and echo server.

EX No.: 8. b)

Date: **Applications using TCP sockets like chat**

Aim:

To write a program in Java to implement an application using TCP sockets like chat.

Algorithm:

1. Start the program.
2. In Server
    a. Create a server socket and bind it to a port.
    b. Listen for a new connection and, when a connection arrives, accept it.
    c. Read the client's message and display it.
    d. Get a message from the user and send it to the client.
    e. Repeat steps 3 and 4 until the client sends "end".
    f. Close all streams.
    g. Close the server and client sockets.
    h. Stop.
3. In Client
    a. Create a client socket and connect it to the server's port number.
    b. Get a message from the user and send it to the server.
    c. Read the server's response and display it.
    d. Repeat steps 2 and 3 until the chat is terminated with the "end" message.
    e. Close all input/output streams.
    f. Close the client socket.
    g. Stop.
4. Stop the program

Program: tcpchatserver.java

```
import java.io.*;
import java.net.*;
class tcpchatserver {
  public static void main(String args[]) throws Exception {
    PrintWriter toClient;
    BufferedReader fromUser, fromClient;
    try {
      ServerSocket Srv = new ServerSocket(4000);
      System.out.print("\nServer started\n");
      Socket Clt = Srv.accept();
      System.out.println("Client connected");
      toClient = new PrintWriter(new BufferedWriter(new
OutputStreamWriter(Clt.getOutputStream())), true);
      fromClient = new BufferedReader(new InputStreamReader(Clt.getInputStream()));
      fromUser = new BufferedReader(new InputStreamReader(System.in));
```

3112410401

```
            String CltMsg, SrvMsg;
            while (true) {
               CltMsg = fromClient.readLine();
               if (CltMsg == null || CltMsg.equalsIgnoreCase("end"))
                  break;
               System.out.println("Client : " + CltMsg);
               System.out.print("Message to Client : ");
               SrvMsg = fromUser.readLine();
               toClient.println(SrvMsg);
               if (SrvMsg != null && SrvMsg.equalsIgnoreCase("end"))
                  break;
            }
            System.out.println("\nClient Disconnected");
            fromClient.close();
            toClient.close();
            fromUser.close();
            Clt.close();
            Srv.close();
         } catch (Exception E) {
            System.out.println(E.getMessage());
         }
      }
   }
```

Program: tcpchatclient.java

```
   import java.io.*;
   import java.net.*;
   class tcpchatclient {
      public static void main(String args[]) throws Exception {
         Socket Clt;
         PrintWriter toServer;
         BufferedReader fromUser, fromServer;
         try {
            Clt = new Socket(InetAddress.getLocalHost(), 4000);
            toServer = new PrintWriter(new BufferedWriter(new
   OutputStreamWriter(Clt.getOutputStream())), true);
            fromServer = new BufferedReader(new InputStreamReader(Clt.getInputStream()));
            fromUser = new BufferedReader(new InputStreamReader(System.in));
            String CltMsg, SrvMsg;
            System.out.println("Type \"end\" to Quit");
            while (true) {
               System.out.print("Message to Server : ");
               CltMsg = fromUser.readLine();
               toServer.println(CltMsg);
```

311241040 15

```
            if (CltMsg == null || CltMsg.equalsIgnoreCase("end"))
               break;
            SrvMsg = fromServer.readLine();
            if (SrvMsg == null || SrvMsg.equalsIgnoreCase("end"))
               break;
            System.out.println("Server : " + SrvMsg);
         }
         System.out.println("\nDisconnected");
         toServer.close();
         fromServer.close();
         fromUser.close();
         Clt.close();
      } catch (Exception E) {
         System.out.println(E.getMessage());
      }
   }
}
```

Output:

```
E:\asen_CN>javac tcpchatserver.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

E:\asen_CN>java tcpchatserver.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

Server started
Client connected
Client : hell
Message to Client : hi
```

```
E:\asen_CN>javac tcpchatclient.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

E:\asen_CN>java tcpchatclient.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Type "end" to Quit
Message to Server : hell
Server : hi
Message to Server :
```

Result:

Thus, a program in Java implemented an application using TCP sockets like chat.

EX No.: 9

Date: **HTTP web client program to download a web page using TCP sockets**

Aim:

To write a program in Java to create an HTTP web client program to download a web page using TCP sockets.

Algorithm:

1. Start.
2. Read the file to be downloaded from the web page.
3. To download an image, use the Java URL class found in the java.net package.
4. The file is downloaded from the server and stored in the current working directory.
5. Stop

Program:

```
import java.io.*;
import java.net.URL;

public class Download {
   public static void main(String[] args) throws Exception {
      try {
         String fileName = "digital_image_processing.jpg";
         String website = "https://www.tutorialspoint.com/java_dip/images/" +
               fileName;
         System.out.println("Downloading File From: " + website);
         URL url = new URL(website);
         InputStream inputStream = url.openStream();
         OutputStream outputStream = new FileOutputStream(fileName);
         byte[] buffer = new byte[2048];
         int length = 0;
         while ((length = inputStream.read(buffer)) != -1) {
            System.out.println("Buffer Read of length: " + length);
            outputStream.write(buffer, 0, length);
         }
         inputStream.close();
         outputStream.close();
         System.out.println("Download complete: " + fileName);
      } catch (Exception e) {
         System.out.println("Exception: " + e.getMessage());
      }
   }
}
```

3112410401015

Output:

```
E:\asen_CN>java Download.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Download.java:11: warning: [deprecation] URL(String) in URL has been deprecated
            URL url = new URL(website);
                      ^
1 warning
Downloading File From: https://www.tutorialspoint.com/java_dip/images/digital_image_processing.jpg
Buffer Read of length: 2048
Buffer Read of length: 2048
Buffer Read of length: 2048
Buffer Read of length: 2048
Buffer Read of length: 2048
Buffer Read of length: 2048
Buffer Read of length: 2048
Buffer Read of length: 2048
Buffer Read of length: 2048
```

211124104015

Result:

Thus, a Java HTTP web client program to download a web page using TCP sockets was written and executed successfully.